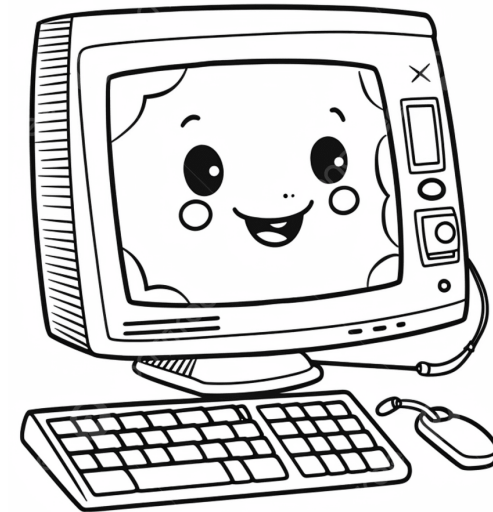
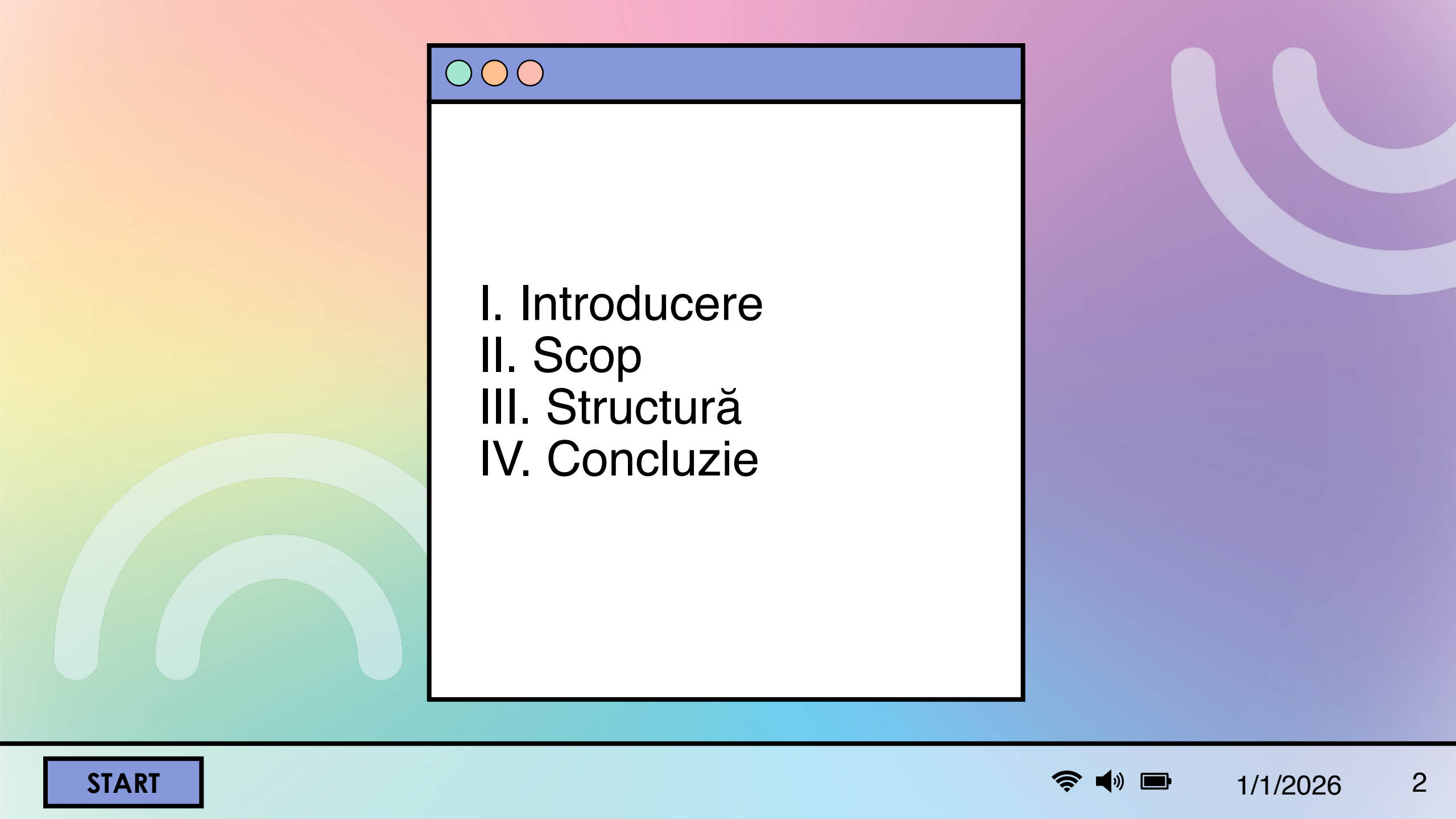


PackageMonitor Extentions

Proiect realizat de:

Croitoru Luiza
Dobre Radu
Teodorescu Luca





I. Introducere
II. Scop
III. Structură
IV. Concluzie

I. Introdurre

Cerința proiectului

Proiectul presupune crearea unui script shell pe un sistem Linux, împărțit în două componente principale, pentru a monitoriza și organiza operațiile de instalare și eliminare a pachetelor software, folosind log-ul sistemului (fișierul **/var/log/dpkg.log**).

Componentele esențiale vor fi:

- **Monitorul** → având ca scop să parseze continuu fișierul **/var/log/dpkg.log** și să extragă informațiile despre operațiile de **install/remove** și să organizeze datele extrase într-un director de lucru (**\$WORK_DIR**)
- **Front-end-ul** → având ca scop să citească informațiile organizate din directorul de lucru și să ofere utilizatorului 9 funcționalități specifice

Funcționalități

1. **lista pachetelor** software **curent instalate** în sistem și **data** ultimei instalări
2. **lista pachetelor** software care **au fost instalate** cândva în sistem și **data** ultimei înlăturări
3. **istoria operațiilor** efectuate în timp asupra unui anumit pachet software
4. **lista pachetelor** software **instalate/înlăturate într-o perioadă dată** de timp
5. **lista pachetelor instalate parțial** și determina **dacă** până la urmă au fost **instalate corect**
6. detectează **dacă** un pachet a fost **instalat pentru prima** dată în sistem
7. **tipărește** la cerere **dimensiunea unui pachet instalat**
8. programul menține **suma dimensiunilor** pachetelor instalate în sistem și o **actualizează** corespunzător cu fiecare install/remove
9. programul **implementează un undo cache** care reține **cele mai recente N remove-uri** și le **afișează** la cerere pentru a putea fi eventual reinstalate

Ce este /var/log/dpkg.log?

Este un fișier jurnal (log) crucial pe sistemele de operare bazate pe **Debian** (inclusiv **Ubuntu**) care utilizează sistemul de gestionare a pachetelor **dpkg**. Înregistrează evenimente legate de:

- **Instalare** (install)
- **Actualizare** (upgrade)
- **Înlăturare** (remove, purge)
- **Configurare** (configure)
- **Dezasambalarea** (unpack) pachetelor software

Cum arată informația din fișier?

```
2025-10-19 17:37:47 install tree:amd64 <none> 2.1.1-2ubuntu3
2025-10-19 17:37:47 status half-installed tree:amd64 2.1.1-2ubuntu3
2025-10-19 17:37:47 status triggers-pending man-db:amd64 2.12.0-4build2
2025-10-19 17:37:47 status unpacked tree:amd64 2.1.1-2ubuntu3
2025-10-19 17:37:47 startup packages configure
2025-10-19 17:37:47 configure tree:amd64 2.1.1-2ubuntu3 <none>
2025-10-19 17:37:47 status unpacked tree:amd64 2.1.1-2ubuntu3
```

Fiecare linie din acest log este compusă din 6 câmpuri separate prin spații, după cum urmează:

(\$1) 2025-10-19	Data când a avut loc evenimentul
(\$2) 17:37:47	Ora evenimentului
(\$3) install, status, startup, configure	Acțiunea - criteriu pentru filtrare
(\$4) tree:amd64	Numele pachetului afectat, incluzând uneori arhitectura (:amd64)
(\$5) <none> sau o versiune	Versiunea Veche (sau starea precedentă)
(\$6) O versiune sau <none>	Versiunea Nouă (sau starea finală)

Pași generali de parsare

1. **Citirea fișierului:** Scriptul monitorului va citi fișierul /var/log/dpkg.log linie cu linie (folosind comenzi de tip cat și bucle while read).
2. **Identificarea câmpurilor:** Pe fiecare linie, vom folosi comenzi ca awk sau cut (și, probabil, grep pentru filtrare) pentru a separa diferitele câmpuri (data, ora, operația, numele pachetului, versiunile) care sunt separate de spații.
3. **Filtrarea:** Vom filtra liniile care conțin operațiile de interes (în principal install și remove).
4. **Extragerea datelor cheie**
5. **Organizarea:** Folosirea datelor extrase pentru a crea structura de fișiere și directoare cerută în directorul de lucru

MODEL:

```
awk ($3 == "install") || ($3 == "remove")  
{ DATA=$1 ORA=$2 OPERATIE=$3 PACHET=$4 VERSIUNE_VECHE=$5 VERSIUNE_NOUA=$6
```


II. Scop

Nu este doar un exercițiu academic, ci un instrument de audit și monitorizare foarte practic. Principalele sale utilități în mediul de producție sunt:

- **Detecția anomaliilor:** Monitorizarea fișierului ajută la detectarea instalărilor neautorizate de pachete care ar putea fi problematice
- **Audit Trail:** Oferă o istorie precisă, iar în cazul unui incident de securitate, poți verifica rapid ce pachete au fost instalate sau modificate înainte de atac
- **Revenire (Rollback) Rapidă:** Dacă un administrator elimină un pachet critic din greșeală, cache-ul îi arată imediat comanda de reinstalare
- **Diagnosticarea Problemelor:** Dacă o aplicație începe să se comporte ciudat după o actualizare, se poate folosi **istoricul operațiilor** (cerința 3) pentru a izola rapid pachetul care a fost instalat sau actualizat chiar înainte de apariția problemei.
- **Instalări Parțiale (Cerința 5):** Detectarea pachetelor blocate în starea de instalare parțială (half-installed) este crucială, deoarece aceste erori pot bloca operațiunile ulterioare de apt/dpkg pe întregul sistem.
- **Gestiunea Spațiului pe Disc (Cerințele 7 și 8)**

III. Structură

Descrierea componentelor cheie (arborele de fişiere)

```
.
├── PackageMonitor/
│   ├── monitor.sh
│   ├── frontend.sh
│   └── helper_functions.sh
└── /var/local/PackageMonitor/ (Directorul de Lucru: $WORK_DIR)
    ├── index_data/
    │   ├── 2025-12-05_installs.txt
    │   └── 2025-12-05_removes.txt
    ├── monitor_status.dat
    ├── total_size.dat
    ├── undo_cache.log
    ├── pachet_A:amd64/
    │   ├── istoric.log
    │   └── stare_curenta.dat
    ├── pachet_B:amd64/
    │   ├── istoric.log
    │   └── stare_curenta.dat
```

Descrierea componentelor cheie

1. Scripturi

monitor.sh (Monitorul):

- Citește /var/log/dpkg.log (pentru o perioadă dată).
- Parsează liniile
- Creează subdirectoarele pachetului (pachet_X:amd64/)
- Actualizează istoric.log, stare_curenta.dat, index_data/ și total_size.dat pentru fiecare pachet

frontend.sh (Front-end-ul):

- Afișează meniul de opțiuni (1-9).
- Citește datele doar din directorul de lucru (/var/local/PackageMonitor/).

Descrierea componentelor cheie

2. Directorul de lucru (\$WORK_DIR)

Acesta este elementul central, servind ca bază de date a sistemului:

- **pachet_X** (Subdirectoare): Permite Front-end-ului să găsească rapid toate informațiile despre un pachet specific
- **stare_curenta.dat**: Fișier crucial care reține starea finală (installed sau removed), data asociată și alte metadata (dimensiunea, linia originală de log)
- **index_data/**: Permite Front-end-ului să răspundă la Cerința 4 fără a scana sute de subdirectoare de pachete.
- **undo_cache.log**: Va conține o listă simplă de N pachete recent eliminate, gestionată după logica LRU (Least Recently Used).
- **total_size.dat**: Un fișier simplu ce stochează suma tuturor dimensiunilor pachetelor instalate.

Structura de meniu cu case în Scriptul Shell

```
function afiseaza_meniu() {
echo "=====
echo " PackageMonitorExtentions - Front-end Meniu"
echo "=====
echo "1. Lista pachetelor curent instalate (si data ultimei instalari)"
echo "2. Lista pachetelor care au fost inlaturate (si data ultimei inlaturari)"
echo "3. Istoria operatiilor asupra unui pachet (la cerere)"
echo "4. Lista pachetelor instalate/inlaturate intr-o perioada data"
echo "5. Detecteaza pachetele instalate partial"
echo "6. Detecteaza pachetele instalate pentru prima data"
echo "7. Tipareste dimensiunea instalata a unui pachet (la cerere)"
echo "8. Afiseaza suma totala a dimensiunilor instalate"
echo "9. Afiseaza Undo Cache-ul (cele mai recente N remove-uri)"
echo "0. Iesire (Exit)"
echo "-----"
read -p "Alege optiunea (0-9): " OPTIUNE }
```

Structura de meniu cu case în Scriptul Shell

```
while true;
do afiseaza_meniu
case $OPTIUNE in
1) lista_pachete_instalate ;;
2) lista_pachete_eliminate ;;
3) istoric_pachet "$PACHET_VIZAT" ;;
.....
9) afiseaza_undo_cache ;;
0) echo "Iesire din program. La revedere!" break ;;
*) echo "Optiune invalida. Va rugam introduceti o cifra intre 0 si 9." ;;
esac
read -p "Apasati [ENTER] pentru a continua..."
done

function lista_pachete_instalate()
{..... }
```

IV. Conclusie

Cazuri particulare

1. Validarea Mediului și a Fișierelor de Log:

- Scriptul monitor.sh trebuie să verifice dacă fișierul /var/log/dpkg.log există și dacă utilizatorul are permisiuni de citire.
- În cazul în care log-ul nu conține nicio înregistrare, monitorul ar trebui să afișeze un mesaj informativ.

2. Gestionarea Integrității Datelor în Monitor:

- Dacă o linie nu respectă formatul de 6 câmpuri, scriptul de parsare trebuie să sară peste acea linie pentru a evita erorile de procesare

Cazuri particulare

3. Logica Funcționalităților Front-end:

- La Cerința 3 (Istoric), dacă utilizatorul introduce un nume de pachet care nu se află în \$WORK_DIR, programul trebuie să afișeze "Pachetul nu a fost găsit" în loc de o eroare de sistem.
- Pentru Cerința 4, trebuie validat formatul datei introduse de utilizator. De asemenea, trebuie gestionat cazul în care data de "sfârșit" este calendaristic înaintea datei de "început".

Mulțumim pentru atenția
acordată !